

Applying Deep Learning to Stock Trading: LSTM Forecasting and Deep Reinforcement Learning

Bicheng Wang

Stanford University

bichengw@stanford.edu

Xinyi Zhang

Stanford University

xyzh@stanford.edu

June 8, 2021

Abstract

Consistently profitable trading is difficult because equity markets are noisy, non-stationary, and only partially predictable from historical data. We compare two deep learning approaches to building a stock trading portfolio. The first is indirect: a two-layer long short-term memory (LSTM) network forecasts prices, and a simple rule converts the forecasts into long or cash positions. The second is direct: deep reinforcement learning (DRL) agents—A2C, PPO, DDPG, TD3, and SAC—learn trading actions end to end from the market state. We use daily data for 20 large-capitalization S&P 500 stocks from 2000 to 2020, augmented with trend-following technical indicators and log min–max scaling; we train on 2000–2018 and backtest on 2019–2020. For forecasting, a mean absolute percentage error (MAPE) loss matches or outperforms MSE, Huber, and log-cosh losses on all validation metrics. In backtesting, DDPG attains the highest annual return (55.5%) and Sharpe ratio (1.38), whereas the LSTM strategy has the smallest maximum drawdown (−9.6%) and the lowest market beta (0.31); with leverage, it reaches an alpha of 0.20, second only to DDPG. Rolling retraining improves A2C, PPO, and SAC, leaves TD3 essentially unchanged, and substantially degrades DDPG, consistent with TD3’s greater robustness to hyperparameters.

Keywords: stock trading, LSTM, deep reinforcement learning, time-series forecasting, portfolio management

1 Introduction

Profitable trading is central to investment management. Because the stock market is dynamic and complex, it is difficult to profit from trading consistently. Machine learning offers clear advantages in data mining, forecasting, and automated trading, and in this work we explore how deep learning can be used to construct a profitable stock portfolio.

We study two complementary approaches (Fig. 1). The *indirect* approach uses time-series forecasting: an LSTM network predicts the market price, and a basic trading strategy acts on the prediction. The *direct* approach uses deep reinforcement learning (DRL): an agent observes the market and outputs trading actions end to end to build the portfolio. Our contributions are as follows:

- We construct a dataset of 20 large-capitalization S&P 500 stocks over 2000–2020, enriched with trend-following technical indicators and log min–max scaling (Section 3).
- We study loss functions, architectures, and training-data regimes for LSTM price forecasting, and find that a MAPE loss gives the lowest validation error among the losses we compare (Section 4.1).
- We benchmark five DRL algorithms (A2C, PPO, DDPG, TD3, and SAC), each with and without rolling retraining, against the LSTM strategy on a 2019–2020 out-of-sample backtest (Sections 4.2 and 5).

2 Related Work

Our use of LSTM networks [1] to predict stock prices was inspired by [2]. Moghar and Hamiche [3] summarize design experience with LSTM recurrent networks for stock market prediction, and the TensorFlow time-series tutorial [4] provides an architectural reference for forecasting models.

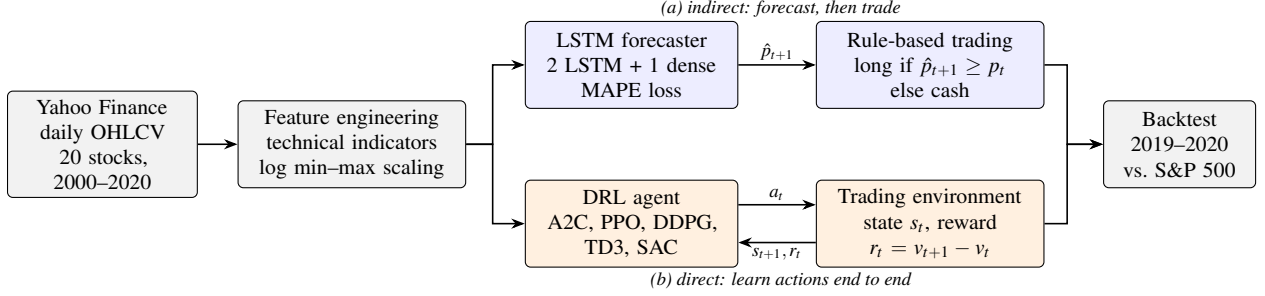


Figure 1: Overview of the two approaches. (a) An LSTM forecasts prices and a rule-based strategy turns the forecasts into positions. (b) A DRL agent interacts with a trading environment and learns trading actions end to end. Both are trained on 2000–2018 and backtested on 2019–2020.

Reinforcement learning for equity trading has attracted growing attention in recent years [5–10], including ensemble DRL strategies for automated stock trading [5], surveys of RL in financial markets [6], and interpretable attention-based portfolio agents [7]. Several challenges remain. (i) Real-world trading data are limited. (ii) The reward of a trading strategy can be defined in many ways, requiring trade-offs among a robust learning rule, the final optimization target, and the limitations of the dataset. (iii) Data sparsity [11]: because training relies solely on historical data, patterns that occurred rarely in the past may not be captured.

3 Dataset and Features

We select the 20 stocks with the largest market capitalization among S&P 500 constituents and collect their daily data from 2000 to 2020. We restrict the universe to these stocks because long, complete historical records are limited and these companies account for a large share of the index’s total market capitalization. Companies that went public after 2000 are excluded so that every stock has complete data over the entire period. Raw data are obtained from the Yahoo Finance API. Each record contains the date, open, high, low, and close prices, trading volume, ticker symbol, and day of the week. The dataset contains 105,680 rows in total (20 stocks $\times$ 5,284 trading days). We split the data chronologically into a training set (2000–2018) and a test set (2019–2020), approximately a 90/10 split. Table 1 shows sample records.

Table 1: Sample of the raw data. *Close* is the split- and dividend-adjusted closing price, which is why it can lie outside the unadjusted low–high range.

Date	Open	High	Low	Close	Volume	Ticker
2000-01-03	0.936384	1.004464	0.907924	0.859423	535,796,800	AAPL
2000-01-03	16.812500	16.875000	16.062500	16.274673	7,384,400	ADBE
2000-01-03	81.500000	89.562500	79.046875	89.375000	16,117,600	AMZN
2000-01-03	25.125000	25.125000	24.000000	13.952057	13,705,800	BAC
2000-01-03	36.500000	36.580002	34.820000	35.299999	875,000	BRK-B

Feature engineering is a crucial step in training a high-quality model: it converts the raw data into a model-ready state. We first check for missing data and then construct the features described below.

3.1 Technical Indicators

In practice, a trader takes many sources of information into account, such as historical prices, current holdings, and technical indicators. We add seven trend-following technical indicators: moving average convergence divergence (MACD), relative strength index (RSI), Bollinger bands (BOLL), commodity channel index (CCI), directional movement index (DX), simple moving average (SMA), and exponential moving average (EMA), several of them computed over multiple look-back windows (see Section 4.2.1 for the full list). Table 2 shows a sample of the processed dataset.

Table 2: Sample of the processed dataset (first trading day; intermediate columns omitted).

Ticker	Day	macd	boll_ub	...	cci_10	dx_30	close_120_sma	close_120_ema
AAPL	0	0	0.9256	...	-66.67	100	0.859	0.859
ADBE	0	0	0.9256	...	-66.67	100	16.274	16.274
AMZN	0	0	0.9256	...	-66.67	100	89.375	89.375
BAC	0	0	0.9256	...	-66.67	100	13.952	13.952
BRK-B	0	0	0.9256	...	-66.67	100	35.299	35.299

3.2 Logarithmic Scaling

Over the long run, the U.S. equity market has grown at a roughly exponential rate despite intermittent recessions, as the century-long history of the S&P 500 illustrates. Exponential growth introduces nonlinearity into price-level features, and features such as the price itself, SMAs, and EMAs are better modeled on a logarithmic scale. We therefore apply log min-max scaling to these features,

$$\tilde{x} = \frac{\log x - \min(\log x)}{\max(\log x) - \min(\log x)}, \quad (1)$$

which makes their patterns easier for a relatively shallow network to capture.

4 Methods

We investigate two approaches to optimizing stock trading strategies. We first combine LSTM time-series forecasting with a simple single-stock trading strategy, and then use DRL models to optimize trading directly.

4.1 LSTM Time-Series Forecasting

The LSTM approach first forecasts the market and then applies a simple strategy to build a portfolio. We designed this forecasting-and-trading pipeline end to end.

4.1.1 Loss Function

The choice of loss function and evaluation metric strongly influences what a model ultimately optimizes, and for stock prediction the appropriate loss differs from that of a generic regression problem. We want predictions to be close to the true values, but “close” must be measured appropriately. Consider AAPL in 2000, when a share was worth about \$1: a \$0.10 increase is a 10% return. In 2021, the equivalent event is a \$200 share rising to \$220. Mean squared error (MSE) on price levels weights the second error $(20/0.1)^2 = 40,000$ times more heavily than the first, even though both represent the same 10% move. MSE therefore underweights low-price periods and produces an unbalanced fit. We compared MSE, mean absolute percentage error (MAPE), mean squared logarithmic error (MSLE), Huber loss, and log-cosh loss.

MAPE better captures the relationship between predicted and actual prices over long periods in which price levels change substantially; in our experiments, models trained with MAPE showed less prediction skew as prices rose. With actual prices y_i and predictions $\hat{y}_i$,

$$\mathcal{L}_{\text{MAPE}} = \frac{100}{n} \sum_{i=1}^n \left| \frac{\hat{y}_i - y_i}{y_i} \right|, \quad (2)$$

whose derivative with respect to each prediction is

$$\frac{\partial \mathcal{L}_{\text{MAPE}}}{\partial \hat{y}_i} = \begin{cases} -\frac{100}{n|y_i|} & \text{if } \hat{y}_i < y_i, \\ \text{undefined} & \text{if } \hat{y}_i = y_i, \\ \frac{100}{n|y_i|} & \text{if } \hat{y}_i > y_i. \end{cases} \quad (3)$$

Table 3: Training and validation metrics of the LSTM variants. *Loss* is each model’s training objective, which is MAPE except for the MSE, Huber, and log-cosh models. Metrics are computed on scaled prices; the subscript *val* denotes validation. Best value in each metric column in bold.

Model	Loss	MAE	MAPE	MSE ($\times 10^{-6}$)	MAE _{val}	MAPE _{val}	MSE _{val} ($\times 10^{-6}$)
Baseline	1.052367	0.004578	1.052367	38	0.004099	0.972073	32
Dropout	2.371855	0.011330	2.371855	271	0.007442	1.535101	95
MSE	0.000336	0.006153	1.412530	336	0.004355	1.036787	38
Huber	0.000098	0.005407	1.253396	196	0.004077	0.974119	35
Log-cosh	0.000066	0.005286	1.226172	133	0.003798	0.898953	29
MAPE	0.935923	0.004015	0.935923	31	0.003789	0.888290	29
Single ticker	2.464035	0.010135	2.464035	172	0.010011	2.582896	145
Fewer units	0.995828	0.004298	0.995828	35	0.003565	0.856291	27
More units	1.074033	0.004663	1.074033	39	0.005058	1.143134	44

Each sample therefore contributes a gradient of constant magnitude that is inversely proportional to its price level, so low-price periods are not underweighted. A squared percentage error, whose gradient scales with the size of the error, is a natural variant when smoother gradients near the optimum are desired.

4.1.2 Architecture and Hyperparameters

Architecture. We explored bidirectional LSTMs, stacked LSTMs with additional layers, and LSTMs followed by multiple dense layers of various sizes. A two-layer LSTM followed by a single dense layer yielded the most stable predictive model.

Units and dropout. At our dataset scale, too many parameters lead to overfitting; mitigating this with dropout or other regularization increased computational cost without improving the metrics (Table 3).

Training data. With 20 stocks, several training regimes are possible: (i) train and predict on a single stock; (ii) train on all stocks and predict a single stock; and (iii) train on all stocks, fine-tune on the target stock, and then predict. Training on all stocks already yields good predictions (compare *Baseline* with *Single ticker* in Table 3), although the model can optionally be fine-tuned on a specific stock before prediction.

Table 3 compares the models we explored.

4.1.3 Trading Strategy

Given the LSTM forecasts, we apply a simple modified buy-and-hold strategy. If the predicted price is lower than the current price, the strategy holds cash; if it is higher than or equal to the current price, it buys shares and holds them. Formally, the position on day t is

$$w_t = \lambda \cdot \mathbb{I}[\hat{p}_{t+1} \geq p_t], \quad (4)$$

where p_t is the current price, $\hat{p}_{t+1}$ is the predicted price, and $\lambda = 1$ for the unleveraged strategy and $\lambda > 1$ for the leveraged one. We evaluate both variants (Table 4).

4.2 Deep Reinforcement Learning

The DRL approach aims to generate portfolio trading actions directly, end to end, from the market environment. We formulate trading as a Markov decision process.

4.2.1 Model Definition

Action. The action space describes how the agent interacts with the environment. For a single stock, a basic action takes one of three values, $a \in \{-1, 0, 1\}$, representing selling, holding, and buying one share. More generally, an action

may involve multiple shares, so for each stock we use the action space $a \in \{-k, \dots, -1, 0, 1, \dots, k\}$, where k is the maximum number of shares per trade. For example, “buy 10 shares of AAPL” and “sell 10 shares of AAPL” correspond to $a = 10$ and $a = -10$, respectively.

Reward. The reward function $r(s, a, s')$ is the incentive for the agent to learn better actions. It is the change in portfolio value when action a is taken in state s , leading to the new state s' :

$$r(s, a, s') = v' - v, \quad (5)$$

where v and v' are the portfolio values at states s and s' , respectively.

State. The state space describes the observations the agent receives from the environment. Just as a human trader analyzes many sources of information before executing a trade, our agent observes a range of features in order to learn in the interactive environment. The state is $s = [b, p, h, \mathbf{x}]$, where b is the available cash balance, p the closing prices, h the number of shares held of each stock, and $\mathbf{x}$ the technical indicators of Section 3.1: `macd`, `boll_ub`, `boll_lb`, `rsi_10`, `rsi_20`, `cci_10`, `cci_20`, `dx_30`, `close_20_sma`, `close_60_sma`, `close_120_sma`, `close_20_ema`, `close_60_ema`, and `close_120_ema`. The numeric suffix of each indicator denotes its look-back window in trading days.

4.2.2 Learning Algorithms

We evaluate five DRL algorithms.

Proximal Policy Optimization (PPO). PPO [12] is a policy-gradient method that limits the size of each policy update. It simplifies trust region policy optimization (TRPO) [13], which enforces a hard constraint on the KL divergence between successive policies, by instead optimizing a clipped surrogate objective (or, in a variant, adding a KL-divergence penalty). This keeps policy learning stable using only first-order optimization. Because each batch of collected experience is reused for several epochs of minibatch updates, PPO is relatively sample efficient, which is valuable when historical trading data are scarce.

Advantage Actor-Critic (A2C). Whereas PPO optimizes the policy directly, value-based methods, which estimate the expected return, can also improve learning. Actor-critic methods [14, 15] combine the two: an actor updates the policy while a critic estimates the value function. We use A2C, the synchronous variant of A3C [16], whose advantage estimate $A(s, a) = r + \gamma V(s') - V(s)$ reduces the variance of policy-gradient updates.

Deep Deterministic Policy Gradient (DDPG). DDPG [17] is an off-policy actor-critic algorithm designed for continuous action spaces, which suits the large, ordered share-count action space. It concurrently learns a Q-function and a deterministic policy: the Q-function is learned from off-policy data via the Bellman equation, and the policy is learned by maximizing the Q-function. Our DDPG implementation builds on Stable-Baselines3 [18].

Twin Delayed DDPG (TD3). In our experiments, DDPG sometimes performed very well but was frequently brittle with respect to hyperparameters and other tuning choices, which caused instability. TD3 [19] addresses these issues with three modifications: it learns two Q-functions and uses the smaller of the two Q-values to form the targets in the Bellman error loss (clipped double-Q learning); it updates the policy less frequently than the Q-functions (delayed policy updates); and it adds noise to the target action to smooth the Q-function (target policy smoothing).

Soft Actor-Critic (SAC). SAC [20] optimizes a stochastic policy in an off-policy manner, bridging stochastic policy optimization and DDPG-style approaches. Its central feature is entropy regularization: the policy is trained to maximize a trade-off between expected return and entropy, which encourages exploration and helps prevent premature convergence to a poor local optimum.

4.2.3 Rolling Training

We compare standard training with rolling training. In rolling training, the test period is divided into consecutive time slices; for each slice, the model is retrained on all historical data available before the slice and then used to trade during that slice.

Table 4: Backtest performance on the 2019–2020 test set; the benchmark is the S&P 500. *Rolling* denotes rolling retraining (Section 4.2.3). Best value in each column in bold (for maximum drawdown, the smallest decline); beta is a measure of market exposure and is not ranked.

Model	Annual return	Cumulative return	Sharpe ratio	Max drawdown	Alpha	Beta
LSTM	16.99%	36.85%	1.19	−9.57%	0.09	0.31
LSTM (leveraged)	34.25%	80.22%	1.19	−19.14%	0.20	0.61
A2C	20.47%	45.23%	0.82	−31.11%	−0.03	1.02
A2C (rolling)	29.83%	68.73%	0.99	−33.52%	0.03	1.14
PPO	27.89%	63.71%	1.09	−25.33%	0.05	0.91
PPO (rolling)	32.16%	74.84%	1.30	−22.84%	0.10	0.84
DDPG	55.51%	142.26%	1.38	−33.52%	0.22	1.24
DDPG (rolling)	29.82%	68.72%	1.09	−33.52%	0.04	1.02
TD3	42.00%	101.93%	1.37	−25.24%	0.18	0.89
TD3 (rolling)	41.89%	101.61%	1.27	−28.68%	0.13	1.11
SAC	38.95%	93.32%	1.33	−24.19%	0.15	0.90
SAC (rolling)	40.44%	97.51%	1.28	−23.20%	0.19	0.83

5 Results

5.1 Evaluation Metrics

We evaluate each strategy on the 2019–2020 test period using annual return, cumulative return [21], Sharpe ratio [22], maximum drawdown [23], alpha, and beta, with the S&P 500 index as the benchmark. The Sharpe ratio measures return per unit of volatility; maximum drawdown is the largest peak-to-trough decline in portfolio value; and alpha and beta are the intercept and slope of a regression of strategy returns on benchmark returns, measuring excess return and market exposure, respectively. Table 4 reports the results.

5.2 Discussion

LSTM. The unleveraged LSTM strategy has the lowest annual return (16.99%), but its Sharpe ratio of 1.19 lies in the middle of the range, and it has the smallest maximum drawdown (−9.57%) and the lowest beta (0.31) of all strategies. Leverage roughly doubles its annual return, maximum drawdown, alpha, and beta while leaving the Sharpe ratio unchanged. The resulting alpha of 0.20 is second only to DDPG (0.22), and the beta of 0.61 remains well below that of every DRL agent (0.83–1.24). Because the LSTM’s price forecasts are easy for humans to interpret, this approach also has strong potential to be combined with other trading strategies.

DRL agents. DDPG attains the highest annual return (55.51%), cumulative return (142.26%), Sharpe ratio (1.38), and alpha (0.22), but also the highest beta (1.24) and one of the deepest drawdowns (−33.52%). TD3 and SAC offer a better balance between return and risk, with Sharpe ratios of 1.37 and 1.33 and maximum drawdowns of −25.24% and −24.19%, respectively.

Rolling training. Rolling retraining improves the annual return of A2C (20.47% → 29.83%), PPO (27.89% → 32.16%), and SAC (38.95% → 40.44%), and leaves TD3 essentially unchanged (42.00% vs. 41.89%). DDPG is the exception: its annual return falls from 55.51% to 29.82%. In our experience, DDPG is difficult to tune to a suitable configuration for every rolling window, which may explain the gap between the individually tuned non-rolling DDPG and its rolling counterpart.

DDPG vs. TD3. Consistent with this observation, TD3 substantially stabilized training in our experiments, making results far less dependent on hyperparameters and tuning, which is a significant advantage in a market that is hard to predict. Its performance is nearly identical with and without rolling retraining.

6 Conclusion

We applied an LSTM time-series forecasting model and five deep reinforcement learning algorithms to learn profitable trading strategies for large-capitalization U.S. equities. For the LSTM, we explored loss functions, architectures, and training regimes to select the best model, and found MAPE to be the most effective loss among those compared. For DRL, we evaluated A2C, PPO, DDPG, TD3, and SAC under two training strategies. On the 2019–2020 test period, the DRL agents achieved the highest returns, led by DDPG with a 55.51% annual return; the LSTM strategy offered the smallest drawdown and market exposure; and TD3 combined strong risk-adjusted performance with stability across training strategies.

Limitations. Our stock universe consists of large companies with complete 2000–2020 histories, which introduces survivorship bias. The test period covers only two years (2019–2020), which include both a strong bull market and the COVID-19 crash of early 2020, so the results may not generalize to other market regimes. Finally, DRL performance is known to be sensitive to random seeds and hyperparameters, and we do not report variability across runs. Future work could address these issues with a point-in-time stock universe, longer walk-forward evaluation, and multiple random seeds.

Author Contributions

Bicheng Wang developed the LSTM forecasting model and half of the reinforcement learning models. Xinyi Zhang developed the other half of the reinforcement learning models.

Acknowledgments

This work was carried out under the guidance of Ayush Kanodia, who advised on comparing the LSTM and reinforcement learning approaches and on the rolling-prediction improvement. We thank Huizi Mao for discussions that inspired the original trading-model design. We also thank Andrew Ng and Kian Katanforoosh, the instructors of Stanford CS230, whose course informed our choices of LSTM hyperparameters, loss functions and their derivatives, evaluation metrics, and reinforcement learning methods.

References

- [1] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [2] Fares Sayah. Stock market analysis + prediction using LSTM. Kaggle notebook. URL <https://www.kaggle.com/faressayah/stock-market-analysis-prediction-using-lstm>.
- [3] Adil Moghar and Mhamed Hamiche. Stock market prediction using LSTM recurrent neural network. *Procedia Computer Science*, 170, 2020. URL <https://www.sciencedirect.com/science/article/pii/S1877050920304865>.
- [4] TensorFlow. Time series forecasting. TensorFlow tutorial, 2020. URL https://www.tensorflow.org/tutorials/structured_data/time_series.
- [5] Hongyang Yang, Xiao-Yang Liu, Shan Zhong, and Anwar Walid. Deep reinforcement learning for automated stock trading: An ensemble strategy. In *Proceedings of the First ACM International Conference on AI in Finance (ICAIF)*, 2020. doi: 10.2139/ssrn.3690996. Available at SSRN: <https://ssrn.com/abstract=3690996>.
- [6] Thomas G. Fischer. Reinforcement learning in financial markets – a survey. FAU Discussion Papers in Economics 12/2018, Friedrich-Alexander-Universität Erlangen-Nürnberg, Institute for Economics, 2018. URL <https://www.econstor.eu/handle/10419/183139>.
- [7] Jingyuan Wang, Yang Zhang, Ke Tang, Junjie Wu, and Zhang Xiong. AlphaStock: A buying-winners-and-selling-losers investment strategy using interpretable deep reinforcement attention networks. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 2019. arXiv:1908.02646.
- [8] Yuqin Dai, Chris Wang, Iris Wang, and Yilun Xu. Reinforcement learning for FX trading. Stanford MS&E 448 project report, 2019. URL http://stanford.edu/class/msande448/2019/Final_reports/gr2.pdf.

- [9] Zhuoran Xiong, Xiao-Yang Liu, Shan Zhong, Hongyang Yang, and Anwar Walid. Practical deep reinforcement learning approach for stock trading. *arXiv preprint arXiv:1811.07522*, 2018.
- [10] Thibaut Théate and Damien Ernst. An application of deep reinforcement learning to algorithmic trading. *Expert Systems with Applications*, 173:114632, 2021. doi: 10.1016/j.eswa.2021.114632.
- [11] Mahdi Nasiri, Behrouz Minaei, and Zeinab Sharifi. Adjusting data sparsity problem using linear algebra and machine learning algorithm. *Applied Soft Computing*, 61, 2017. doi: 10.1016/j.asoc.2017.05.042.
- [12] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [13] John Schulman, Sergey Levine, Philipp Moritz, Michael Jordan, and Pieter Abbeel. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 1889–1897, 2015.
- [14] Vijay R. Konda and John N. Tsitsiklis. Actor-critic algorithms. In *Advances in Neural Information Processing Systems 12 (NIPS)*, pages 1008–1014, 2000.
- [15] David Pfau and Oriol Vinyals. Connecting generative adversarial networks and actor-critic methods. *arXiv preprint arXiv:1610.01945*, 2016.
- [16] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pages 1928–1937, 2016.
- [17] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2016. arXiv:1509.02971.
- [18] Antonin Raffin, Ashley Hill, Maximilian Ernestus, Adam Gleave, Anssi Kanervisto, and Noah Dormann. Stable baselines3. GitHub repository, 2019. URL <https://github.com/DLR-RM/stable-baselines3>.
- [19] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 1587–1596, 2018.
- [20] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 1861–1870, 2018.
- [21] Cumulative return. Investopedia. URL <https://www.investopedia.com/terms/c/cumulativereturn.asp>.
- [22] Sharpe ratio. Investopedia. URL <https://www.investopedia.com/terms/s/sharperatio.asp>.
- [23] Maximum drawdown (MDD). Investopedia. URL <https://www.investopedia.com/terms/m/maximum-drawdown-mdd.asp>.